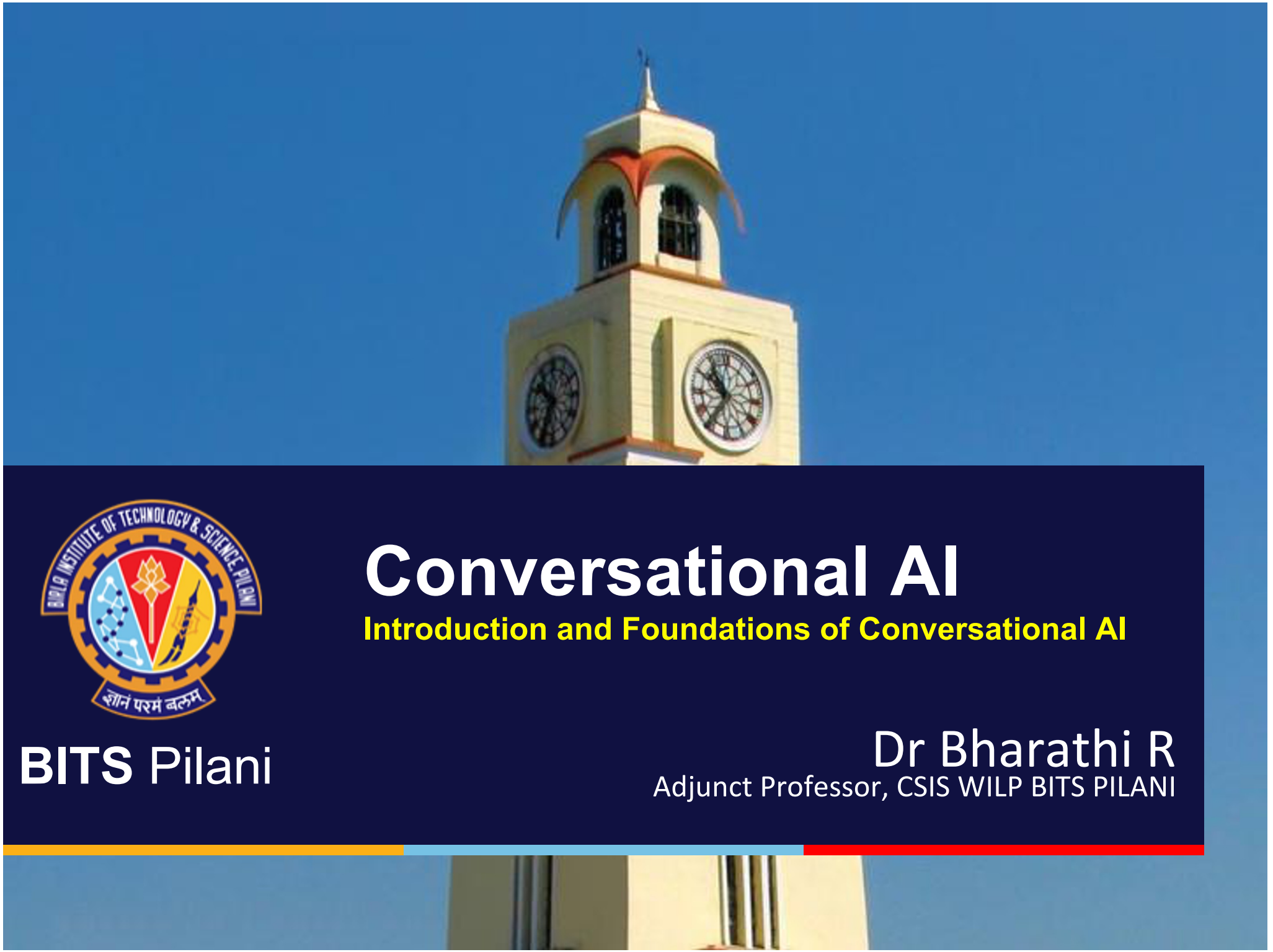


Conversational AI

Introduction and Foundations of Conversational AI

BITS Pilani

Dr Bharathi R
Adjunct Professor, CSIS WILP BITS PILANI

innovate achieve lead

BITS Pilani

AIMLCZG521 , Conversational AI
Lecture No. 1 | Module 1

Disclaimer & Acknowledgement

DISCLAIMER

These instructional materials have been developed by the instructor for educational purposes. All content is intended solely to support the learning objectives of this course. While every effort has been made to ensure accuracy and completeness, learners are encouraged to consult original source publications for authoritative reference.

ACKNOWLEDGEMENT

These instructional materials have been developed by the instructor with due acknowledgement of the original publication authors and the broader academic community for their openly accessible resources, along with special recognition of Prof. Bhagath's Conv-AI-WILP (October 2025) presentation materials, which have been adapted and modified to support the learning objectives of this course.

What Is Conversational AI?

innovate achieve lead

Definition: Any AI system that engages humans through natural language to understand intent, retain context, retrieve knowledge, and deliver information or take real-world action.

Core Capabilities

Natural Language Understanding (NLU)
Understanding user intent and entities

Dialogue Management
Managing conversation flow and context

Natural Language Generation (NLG)
Generating human-like responses

Context Awareness
Remembering conversation history

Multi-turn Interaction
Handling complex dialogues

Personalization
Adapting to user preferences

Understand

Interpret natural language intent, entities, sentiment

Reason

Plan, chain thoughts, decompose multi-step problems

Act

Call APIs, write code, retrieve docs, orchestrate agents

Examples in Daily Life

Voice Assistants: Alexa, Siri, Google Assistant

Customer Support: Banking chatbots, e-commerce help

Virtual Agents: Healthcare triage, HR assistants

The Rise of Conversational AI: Market Trends & Industry Impact

innovate achieve lead

$41.39B

Conv-AI Market by 2030

Source: Grand View Research, 2025

100M

ChatGPT Users in 2 Months

Fastest-growing app in history (Wikipedia / UBS)

80%

Fortune 500 Using AI Agents

Source: Microsoft Copilot Studio Telemetry, Nov 2025

Conversational AI: Industry Applications & Impact

Banking & Finance

70% of Tier-1 queries handled by AI; up to 60% cost reduction

Sources: CoinLaw AI in Banking Stats 2025; AutomationEdge Banking AI Report 2025

Healthcare

AI triaging cuts ER wait times; supports 24/7 remote patient monitoring

Source: McKinsey AI in Healthcare 2025; Healthcare AI Adoption Report

Enterprise Software

Agents autonomously browse, code, draft contracts & operate tools

Source: Gartner Agentic AI Report Q1 2025; DigitalDefynd Agentic AI Stats

E-Commerce

15–26% avg. conversion rate lift via AI personalised recommendations

Sources: MindStudio AI Agents for E-commerce 2026; Barilliance Research

$41.39B by 2030 per GVR press release (May 2025). | 80% Fortune 500 stat: Microsoft first-party telemetry, Nov 2025. | E-commerce conversion range: 15–26% per Barilliance/MindStudio aggregate data.

The Evolution Journey (1960s - 2026)

Era	Technology	Capabilities	Limitations
1960s–1990s	Rule-Based (ELIZA, ALICE)	Pattern matching, keyword detection, simple scripted Q&A	No learning, rigid, no context
2000–2010	Statistical ML (SVMs, CRFs, HMMs)	Intent classification, entity extraction	Limited context, hand-crafted features
2010–2017	Deep Learning (RNNs, LSTMs, Seq2Seq, Attention)	End-to-end learning, better context understanding	Data hungry, task-specific training
2017–2020	Transformers (BERT, GPT-2, T5)	Transfer learning, contextual embeddings	Still requires task-specific fine-tuning
2020–2023	LLMs & Generative AI (GPT-3, ChatGPT, Claude, PaLM)	Few-shot learning, general-purpose, fluent generation	Hallucinations, no real-time data, no actions
2023–2025	Agentic AI (LLMs + Tools + Memory + Planning)	Execute actions, multi-step reasoning, autonomous workflows	Complex orchestration, scaling challenges, cost optimization
2025–2026	On-Device & Multi-Modal (SLMs, native multimodality)	Real-time voice/video, privacy-first local processing	Hardware constraints, fragmented ecosystems

Key driver: Three simultaneous breakthroughs made LLMs possible — the Transformer architecture (2017), affordable GPU compute, and internet-scale training data. Remove any one and we're still in the chatbot era.

Evolution Deep Dive: Early Stages

1. Rule-Based Systems (1960s-1990s)

Example: ELIZA (1966)

```
IF user_input contains "mother": RESPOND "Tell me more about your family"
IF user_input contains "sad": RESPOND "Why do you feel sad?"
```

Limitations: No learning, brittle, couldn't handle variations, no real understanding

Evolution Deep Dive: Early Stages

2. Statistical ML Era (2000-2010)

Key Techniques:

- **Intent Classification:** SVM, Naive Bayes to classify user intent
- **Named Entity Recognition:** CRF, HMM for extracting entities
- **Dialogue State Tracking:** Markov models for conversation flow

Example Framework: Microsoft LUIS, IBM Watson (early versions)

Advance: Could learn from data, handle variations better

Limitations: Required feature engineering, limited context understanding

Evolution Deep Dive: Deep Learning Era

3. Deep Learning Revolution (2010-2017)

Key Architectures:

- **RNNs/LSTMs:** For sequence modeling, context retention
- **Seq2Seq Models:** Encoder-decoder for response generation
- **Attention Mechanism:** Focus on relevant parts of input

Example Frameworks: Rasa (open source), Google Dialogflow

Advance: End-to-end learning, better context, no hand-crafted features

Limitations: Data hungry, task-specific models, limited transfer learning

Evolution Deep Dive: Deep Learning Era

4. Transformer Era (2017-2020)

Breakthrough: "Attention is All You Need" (2017)

- **Pre-trained Models:** BERT, GPT-2, T5, RoBERTa
- **Transfer Learning:** Pre-train on massive data, fine-tune for tasks
- **Contextual Embeddings:** Word meaning depends on context

Impact on Conversational AI:

- Better intent classification with fewer examples
- Improved entity recognition
- More natural response generation

Evolution: Modern Era (2020-2025)

innovate achieve lead

5. Large Language Models (2020-2023)

Game Changers: GPT-3 (2020), ChatGPT (2022), Claude, PaLM

- **Massive Scale:** Billions to trillions of parameters
- **Few-Shot Learning:** Learn tasks from examples in prompt
- **General Purpose:** One model for many tasks
- **Human-like Fluency:** Natural, contextual responses

Revolution for Conversational AI:

- No fine-tuning needed for many tasks
- Understand complex, multi-turn conversations
- Generate creative, contextual responses

But still limited: No real-time data, can't take actions, hallucinations

Evolution: Modern Era (2020-2025)

6. Agentic Conversational AI (2023-2025)

The Next Frontier: LLMs + Tools + Memory + Planning

- **Tool Use:** Can call APIs, search databases, execute code
- **Multi-step Reasoning:** Break down complex tasks
- **Memory Systems:** Remember user preferences, conversation history
- **Autonomous Actions:** Book appointments, send emails, create documents

Evolution : Multi-Agent & Reasoning Systems (2025–early 2026)

innovate achieve lead

- Beyond Agents: Orchestration + Deep Reasoning
- Multi-Agent Frameworks: Agents spawn and supervise sub-agents for parallel task execution
- Extended Thinking: Models reason step-by-step before responding (chain-of-thought at scale)
- Computer Use: AI browses the web, writes and runs code, controls desktop apps
- Long Context Windows: 1M+ token contexts — entire codebases, legal documents, research papers
- Specialised Models: Coding agents (Claude Code, Cursor), science models, domain-specific fine-tunes

AI moves from assistant to autonomous collaborator — working alongside humans on complex, multi-day projects.

Architecture Evolution: Then vs Now

Traditional Architecture (Pre-2020)

User Input ↓ Speech Recognition (if voice) ↓ Natural Language Understanding • Intent Classification • Entity Extraction ↓ Dialogue Manager • State Tracking • Policy (rules/ML) ↓ Natural Language Generation • Template-based • Rule-based ↓ Text-to-Speech (if voice) ↓ Response

Frameworks: Rasa, Dialogflow, Microsoft Bot Framework, Amazon Lex

Modern Agentic Architecture (2023+)

User Input ↓ LLM-based Understanding • Intent + Entities in one pass ↓ Agentic Orchestration Layer • Planning (break down task) • Tool Selection • Memory Retrieval ↓ Tool Invocation • API calls • Database queries • Code execution ↓ Memory Update • Store context • Update user profile ↓ LLM-based Generation • Contextual response ↓ Safety & Validation ↓ Response

Frameworks: LangChain, LlamaIndex, AutoGPT, CrewAI

Use Case: Customer Support Evolution

innovate achieve lead

Scenario: Banking Customer Support

Approach	Capabilities	User Experience
Rule-Based (2000s)	• FAQ matching • Keyword detection • Fixed responses	User: "I lost my card" Bot: "For lost card, press 1. For stolen, press 2" Rigid, frustrating
Intent-Based (2015)	• Intent classification • Entity extraction • Dialogue flow	User: "I lost my card" Bot: "I'll help you block your card. Which card - Credit or Debit?" Better, but limited
LLM-based (2023)	• Natural conversation • Context understanding • Fluent responses	User: "I think I left my card at the restaurant" Bot: "I understand. Let me help you block it temporarily while you check..." Natural, but no action
Agentic AI (2025)	• Everything above + • Access banking system • Execute transactions • Multi-step actions	User: "I lost my card at City Mall" Bot: "I've immediately blocked your card ending in 1234. I see recent transactions at City Mall - last one was $45.20 at Starbucks 2 hours ago. Should I order a replacement card to your home address?" Proactive, action-oriented

Business Impact: Resolution time reduced from 15 minutes (human agent) → 2 minutes (agentic AI). Customer satisfaction increased by 40%.

Components of Modern Conversational AI

1. Natural Language Understanding

Role: Understand what user wants

- Intent classification
- Entity extraction
- Sentiment analysis
- Context understanding

Modern approach: LLM-based, single pass

2. Dialogue Management

Role: Manage conversation flow

- State tracking
- Context maintenance
- Turn-taking
- Error handling

Modern approach: LLM reasoning + Memory systems

Components of Modern Conversational AI

innovate achieve lead

3. Knowledge Access

Role: Retrieve relevant information

- Vector databases
- Semantic search
- RAG (Retrieval-Augmented Generation)
- Real-time data access

Modern approach: Embeddings + Hybrid retrieval

4. Action Execution

Role: Take actions on behalf of user

- Tool/function calling
- API integrations
- Database operations
- External service invocation

Modern approach: Agentic tool use

Components of Modern Conversational AI

innovate achieve lead

5. Response Generation

Role: Generate natural responses

- Contextual generation
- Personality/tone
- Multi-modal output
- Structured responses

Modern approach: LLM generation with control

6. Memory Systems

Role: Remember user context

- Short-term (conversation)
- Long-term (user profile)
- Episodic memory
- Semantic memory

Modern approach: Vector + SQL hybrid

Framework Evolution for Conversational AI

innovate achieve lead

Traditional Frameworks (2015-2020)

Framework	Approach	Use Case
Rasa	Intent + Entity + Dialogue Policies	Custom chatbots, on-premise deployment
Google Dialogflow	Intent-based, ML-powered NLU	Voice assistants, Google integrations
Microsoft Bot Framework	Intent + Entity + Dialogue Management	Enterprise bots, Azure integrations
Amazon Lex	Intent-based, Alexa-powered	Voice + text bots, AWS services

Key Shift: From intent-based dialogue systems to LLM-powered agentic systems with tool use and planning capabilities.

To

Modern Agentic Frameworks (2023-2025)

Framework Evolution for Conversational AI

Modern Agentic Frameworks (2023-2025)

Framework	Approach	Use Case
LangChain / LangGraph	LLM orchestration + Tools + Memory + Workflows	Complex agents, RAG, multi-step reasoning, production systems
LlamaIndex	Data-centric, RAG-optimized pipelines	Document Q&A, knowledge bases, enterprise search
Semantic Kernel	Microsoft's SDK for AI orchestration	Enterprise, .NET/Azure ecosystem integration
Haystack	End-to-end NLP framework	Production RAG systems, search applications
AutoGen	Multi-agent conversation framework	Complex workflows with agent collaboration

State of the Art — 2026

innovate achieve lead

10 M+

Token context window
Llama 4 Scout

94.3 %

GPQA Diamond
Gemini 3.1 Pro (leads reasoning)

5+

Modalities
Text · Image · Audio · Video · Code

Native

Agentic workflows
Browser · OS · Multi-agent · MCP

Leading Models (April 2026)

Model	Provider	Context	Strength	Best For
GPT-5.4	OpenAI	1M	Best all-rounder, computer use	Knowledge work, production APIs
Claude Opus	Anthropic	1M	Coding, safety, long reasoning	Complex agents, coding editors
Gemini 3.1 Pro	Google	1M	Reasoning leader (94.3% GPQA)	Research, multimodal, cost-efficient
Grok 4.20	xAI	128K	Real-time data, multi-agent	Live info, social/market signals
Llama 4 Scout open	Meta	10M	Open-weight, ultra-long context	On-prem / custom deployments
DeepSeek V3.2 open	DeepSeek	128K	~90% of GPT-5.4 at 1/50th cost	Budget-conscious, high-volume API

The Agentic Shift — From Answering to Acting

Traditional Chatbot / LLM		Agentic Conversational AI
Single-turn Q&A responses		Multi-step planning & execution
No access to external tools or data		Tool calling: APIs, search, code, databases
Fixed knowledge cutoff	shift	Real-time data via RAG and web search
No memory across sessions		Persistent memory (vector + SQL stores)
One model, one task		Multi-agent orchestration & delegation

"LLM is the brain — but it needs tools, memory, and planning to become a truly autonomous conversational agent."

Course Architecture — 4 Modules · 16 Lectures

innovate achieve lead

The Paradigm Shift: Conversational AI has fundamentally changed from rule-based systems to intelligent, autonomous agents. This course prepares you for the **current and future** state of conversational AI.

01 Foundations L1–L3

- Conversational AI landscape & agent lifecycle
- Embeddings & vector search (HNSW, ANN, BM25)
- Model landscape: LLMs, MoE, SLMs, SSMs
- Cost engineering: quantization, KV-cache, prompt caching

02 Core Building Blocks L4–L8

- Function calling & ReAct framework
- Fine-tuning: QLoRA, DPO, GRPO
- Agent memory: short-term, long-term, hybrid
- RAG pipelines: chunking, re-ranking, contextual retrieval

03 Autonomous Agents L9–L11

- Agent planning & multi-agent systems
- State management, hierarchical architectures
- RAG & agent evaluation (LLM-as-Judge)
- Cost optimisation & model routing

04 Production Ecosystem L12–L16

- Security: prompt injection, PII, red-teaming
- MCP deep dive: client-server, resources, tools
- A2A protocol & agent interoperability
- Ethics, governance & bias mitigation

What You Will Build — Learning Outcomes

innovate achieve lead

LO1 End-to-End Conversational AI Systems

Design agents with vector databases for retrieval, function calling for tool use, and graph-based workflows for complex reasoning.

LO2 Cost-Effective Production Solutions

Apply prompt caching, model routing, and efficient retrieval strategies that reduce token usage by up to 90%.

LO3 Secure & Observable Agents

Defend against prompt injection, handle PII properly, and build monitoring dashboards tracking performance, costs, and errors.

LO4 Ethical & Governed AI Agents

Mitigate bias, include human oversight for critical decisions, and comply with emerging AI regulations (EU AI Act, Anthropic RSP).

Evaluation Scheme & Important Rules

innovate achieve lead

10%	**EC-1: Quizzes (×2)** Open book	Announced in class
20%	**EC-1: Assignments (×2)** Open book	~3 weeks each
30%	**EC-2: Mid-Term Exam** Closed book (L1–L8)	As per calendar
40%	**EC-3: End-Semester Exam** Open book (L1–L16)	As per calendar

⚠ **Important:** No deadline extensions · No make-ups for quizzes/assignments · Plagiarism = nullified marks + disciplinary action

Pre-requisites

What You Need Coming In

🐍 **Python:**	Functions, classes, basic libraries
🤖 **ML Basics:**	Training, inference, evaluation
🧠 **Deep Learning:**	Neural networks, transformers
🔌 **API Basics:**	REST calls, JSON data handling
📊 **Basic Statistics:**	Precision, recall, probability

Tips

1. **Code every lecture:**
   Don't just watch — run the demo code and change one thing.
2. **Use the references:**
   ReAct, DPR, DPO papers — read the abstract + results at minimum.
3. **Get an API key early:**
   OpenAI or Anthropic. Free tiers cover all demos.
4. **Start assignments early:**
   No extensions. 3 weeks sounds long until it isn't.
5. **Ask on Canvas:**
   Peer discussion is encouraged. Learning happens in community.

Textbooks, Papers & Key Resources

innovate achieve lead

T1 — Official Documentation (Primary Textbooks)

- Anthropic's 'Building Effective Agents' guide — anthropic.com
- OpenAI Function Calling documentation — platform.openai.com
- MCP (Model Context Protocol) & A2A Specification — official specs

R1 — Foundational Reference

- Jurafsky & Martin, 'Speech and Language Processing' (3rd ed.) — NLP foundations, linguistic background

R2 — Core Research Papers (provided during course)

- ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al., 2023)
- Direct Preference Optimization (Rafailov et al., 2023)
- Dense Passage Retrieval for Open-Domain QA (Karpukhin et al., 2020)
- MemGPT: Towards LLMs as Operating Systems (2023)
- MetaGPT: Meta Programming for Multi-Agent Systems (2024)
- The Landscape of AI Agents (2024) — arxiv.org/abs/2404.11584

R3 — Live Resources

- Anthropic Contextual Retrieval (2024) — anthropic.com/news/contextual-retrieval
- Anthropic Prompt Caching — docs.anthropic.com
- Anthropic Responsible Scaling Policy — anthropic.com
- Conference proceedings: ACL, EMNLP, NeurIPS, ICML

Open Problems — Where Research Begins

innovate achieve lead

Consistent multi-step reasoning

LLMs still fail on novel logic tasks outside training distribution. Chain-of-Thought helps but doesn't fully solve it.

Active research: process reward models, tree-of-thought

Persistent cross-session memory

Every new conversation starts blank. External memory is a workaround, not a native solution — lossy, expensive, fragile.

Active research: MemGPT, Titans (Google, 2024)

Grounded factual accuracy

Hallucination: models confidently generate plausible-sounding falsehoods. RAG reduces but doesn't eliminate this at scale.

Active research: attribution, RLHF, verification agents

Reliable long-horizon agent execution

Agents that run 100-step tasks without drifting, getting stuck, or making compounding errors remain elusive.

Active research: agent benchmarks (GAIA, SWE-bench)

Safety & alignment at scale

As agents gain more autonomy, ensuring they follow human intent without unexpected side-effects becomes critical.

Active research: Constitutional AI, RLAIF, interpretability

Compute & energy efficiency

State-of-the-art models require enormous infrastructure. Efficient inference (SSMs, quantization) is an open engineering problem.

Active research: Mamba, QLoRA, mixture-of-experts

Conversational AI

Foundations of Conversational AI
Module-1

BITS Pilani

Dr Bharathi R

Lecture Overview

innovate achieve lead

1 **NLP Foundations**

Tokenization · Context Windows

2 **Transformers & LLMs**

Architecture · Capabilities · Limitations

3 **Chatbots → Agentic Systems**

Evolution · Why Agentic AI?

4 **System Lifecycle & Architecture**

7-Stage Pipeline · Components

5 **Protocol Landscape**

MCP · LangGraph · REST

6 **Production Concerns**

Cost · Latency · Safety

1. Tokenization — The Foundation

innovate achieve lead

What is Tokenization?

Breaking text into subword units (tokens) that LLMs process.
Algorithms: BPE (GPT), SentencePiece (multilingual), WordPiece (BERT)

Text	Tokens	Count
"Hello World"	Hello · World	2 tokens
"artificial intelligence"	art · ificial · intelligence	3 tokens
"GPT-4"	G · PT · - · 4	4 tokens
"Book a flight to NYC"	Book · a · flight · to · NYC	5 tokens

Why It Matters for Conversational AI

💰 **Cost:**
API pricing is per token → impacts conversation economics

🪟 **Context Window:**
Limits conversation length (200K tokens ≈ 150K words)

⚡ **Latency:**
More tokens = slower response in conversations

🎯 **Quality:**
Tokenization affects understanding of domain-specific terms

💡 Key Insight: Model selection & prompt optimization can cut token costs by 10–20× (ref. Lecture 11)

1.Tokenization: Impact on ConvAI

Impact on Conversational AI

Scenario: Customer support conversation (2025 typical costs)

- Average conversation: 40-60 turns (user + agent exchanges)
- Average tokens per turn: 15-25 tokens
- Total per conversation: ~800-1,200 tokens
- Cost with GPT-4o: ~$0.01-0.03 per conversation
- Cost with GPT-3.5 Turbo: ~$0.002-0.005 per conversation
- At 10,000 conversations/day with GPT-4o: $100-300/day

Key insight: Model selection and optimization can reduce costs by 10-20x. We'll cover these techniques in Lecture 11.

Hands-On: Byte-Pair Encoding

innovate achieve lead

Understanding Tokenization with Code

We'll explore how text is converted to tokens using the tiktoken library

Demo Components:

- **Text to Tokens:** See how different texts are tokenized
- **Token Counting:** Calculate tokens for a sample conversation
- **Cost Analysis:** Understand the economic implications
- **Model Comparison:** Tokenization differences across models

Hands-On: Byte-Pair Encoding

• **BPE** is a "subword" tokenization algorithm. It sits in the "Goldilocks" zone between character-based tokenization (too long, little meaning) and word-based tokenization (huge vocabulary, fails on unknown words)

How it works (The Concept):

1.Training: It reads a massive corpus of text and counts adjacent pairs of characters.

2.Merging: It takes the most frequent pair (e.g., 'e' and 'r' -> 'er') and adds it to the vocabulary as a new unit.

3.Iterating: It repeats this thousands of times. Eventually, it learns common prefixes (un-, re-), suffixes (-ing, -ed), and whole common words (apple, table).

BPE training starts by computing the unique set of words used in the corpus (after the normalization and pre-tokenization steps are completed), then building the vocabulary by taking all the symbols used to write those words.

As a very simple example, let's say our corpus uses these five words:

"hug", "pug", "pun", "bun", "hugs"

The base vocabulary will then be ["b", "g", "h", "n", "p", "s", "u"].

Byte-Pair Encoding- first merging

let's assume the words had the following frequencies:

("hug", 10), ("pug", 5), ("pun", 12), ("bun", 4), ("hugs", 5)

meaning "hug" was present 10 times in the corpus,
"pug" 5 times,
"pun" 12 times,
"bun" 4 times, and
"hugs" 5 times.

- We start the training by splitting each word into characters (the ones that form our initial vocabulary) so we can see each word as a list of tokens:

- ("h" "u" "g", 10), ("p" "u" "g", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "u" "g" "s", 5)

- Then we look at pairs. The pair (”u", ”g") is present in the words "hug", "pug", and "hugs", for a grand total of 20 times in the vocabulary.

Byte-Pair Encoding- Second Merging

Thus, the first merge rule learned by the tokenizer is ("u", "g") -> "ug",
which means that "ug" will be added to the vocabulary, and the pair should be merged in all the words of the corpus.
At the end of this stage, the vocabulary and corpus look like this:

Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug"]
Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "u" "n", 12), ("b" "u" "n", 4), ("h" "ug" "s", 5)

Now we have some pairs that result in a token longer than two characters: the pair ("h", "ug"), for instance (present 15 times in the corpus).

The most frequent pair at this stage is ("u", "n"), however, present 16 times in the corpus, so the second merge rule learned is ("u", "n") -> "un".

Adding that to the vocabulary and merging all existing occurrences leads us to:
Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un"]
Corpus: ("h" "ug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("h" "ug" "s", 5)

Byte-Pair Encoding- Third Merging

Now the most frequent pair is ("h", "ug"), so we learn the merge rule ("h", "ug") -> "hug", which gives us our first three-letter token.

After the merge, the corpus looks like this:
Vocabulary: ["b", "g", "h", "n", "p", "s", "u", "ug", "un", "hug"]
Corpus: ("hug", 10), ("p" "ug", 5), ("p" "un", 12), ("b" "un", 4), ("hug" "s", 5)

And we continue like this until we reach the desired vocabulary size.

Let's take the example we used during training, with the three merge rules learned:

("u", "g") -> "ug"
("u", "n") -> "un"
("h", "ug") -> "hug"

The word "bug" will be tokenized as ["b", "ug"]. "mug", however, will be tokenized as ["[UNK]", "ug"] since the letter "m" was not in the base vocabulary.

Likewise, the word "thug" will be tokenized as ["[UNK]", "hug"]: the letter "t" is not in the base vocabulary, and applying the merge rules results first in "u" and "g" being merged and then "h"and "ug" being merged.

How do you think the word "unhug" will be tokenized?

2. Context Windows — Conversation Memory

GPT-4 Turbo 128K tokens (~96K words)	Standard production
Claude 3.5 Sonnet 200K tokens (~ 150K words)	Extended conversations
Gemini 1.5 Pro 1M tokens (~750K words)	Entire codebases
Emerging Models 2M+ tokens	Specialized use cases

⚠ Challenge: 'Lost in the middle' — models struggle with info in long contexts. Solution: RAG + Memory systems (Module 2)

3. Transformers & LLMs — The Brain of Modern Conversational AI

Transformer (2017) → Pre-training on Massive Data → LLM (GPT / BERT / T5) → Fine-tuning / Instruction Tuning → Tools + Memory + Planning → Conversational Agent

Technical Capabilities

Self-Attention:
Understand relationships between words across the full input

Contextual Understanding:
Word meaning depends on surrounding context

Long-Range Dependencies:
Connect ideas across very long text passages

Parallel Processing:
Faster training than older RNN-based architectures

Impact on Conversations

Multi-turn Coherence:
Remember and track earlier parts of a conversation

Intent Understanding:
Grasp user goals from conversational context

Natural Responses:
Generate human-like, fluent replies

Ambiguity Handling:
Resolve unclear or underspecified requests

Key Insight: LLM is the "brain" but needs additional systems (tools, memory, planning) to become a fully functional conversational agent.

3. LLM Capabilities & Limitations of Conversational AI

innovate achieve lead

✅ What LLMs Do Well

- **Natural conversation:** Human-like dialogue
- **Intent understanding:** Grasp user goals
- **Context retention:** Multi-turn conversations
- **Response generation:** Fluent, contextual replies
- **Summarization:** Condense information
- **Entity extraction:** Identify key information
- **Sentiment analysis:** Understand user emotions

✗ LLM Limitations

- **No real-time data:** Training cutoff date
- **Hallucinations:** Generate plausible but false info
- **Can't take actions:** No access to external systems
- **No memory:** Forget after context window
- **Calculation errors:** Struggle with math
- **Consistency:** Different answers to same question
- **No verification:** Can't check facts

3. Why We Need Agentic Architecture

Solution: Augment LLMs with external capabilities

LLM Limitation	Agentic Solution	Example
No real-time data	Tool calling (APIs, search)	Check current flight prices
Hallucinations	RAG (Retrieval-Augmented Generation)	Ground answers in company docs
Can't take actions	Function calling	Book appointment, send email
No memory	External memory systems	Remember user preferences
Calculation errors	Calculator tool, code execution	Precise math computations

4. Agentic Conversational AI — Request → Response Pipeline

How a modern conversational agent processes user input

1 📥 **REQUEST**

Receive & validate input · Parse user intent · Extract entities · Input sanitization (security)

2 🔀 **ROUTING**

Intent classification · Classify intent type · Determine required tools · Route to sub-agent

3 🧠 **REASONING**

Planning & decision making · Break into steps · Identify information gaps · Plan tool call sequence

4 ⚙️ **TOOL INVOCATION**

External API calls & actions · Execute API calls · Database queries · Code execution

5 💾 **MEMORY**

Context storage & retrieval · Store conversation history · Update user preferences · Maintain session state

6 🛡️ **SAFETY**

Guardrails & output validation · Check for toxic content · Verify factual accuracy · PII redaction

7 📤 **RESPONSE**

Final output delivered · Natural language generation · Format for channel (text/voice/UI) · Send to user

4.Lifecycle Example — Banking Conversational Agent

User: "What's my account balance and recent transactions?"

1. Request

Intent: account_inquiry
Entities: {type: [balance, transactions]}
Validate: User authenticated

2. Routing

Route to: Banking Agent
Required tools: get_balance, get_transactions
Permission check: User has access

3. Reasoning

Plan:
1. Fetch account balance
2. Fetch recent transactions (last 5)
3. Format response with both

4. Tool Invocation

Call: get_balance(user_id) → $5,432.10
Call: get_transactions(user_id, limit=5)
Returns: [Transaction1, Transaction2, ...]

5. Memory

Store: User asked about balance at 2:30 PM
Update: User preference for transaction details
Context: Maintain for follow-up questions

6. Safety

Validate: Balance & transactions for correct user
Check: No PII exposure in logs
Verify: Response contains only authorized info

7. Response

Your current balance is $5,432.10. Here are your recent transactions:
1. Starbucks - $4.50 (Today)
2. Amazon - $89.99 (Yesterday)
3. Shell Gas - $45.00 (Yesterday)
4. Grocery Store - $123.45 (2 days ago)
5. Netflix - $15.99 (3 days ago)
Would you like more details on any of these?"

Exercise: Map the Agent Lifecycle

Scenario:

"Find me a good Italian restaurant near my office that's open tonight and make a reservation for 2 at 7 PM"

Map This Request to the 7-Stage Lifecycle

Exercise: Map the Agent Lifecycle

Scenario:

"Find me a good Italian restaurant near my office that's open tonight and make a reservation for 2 at 7 PM"

Stage	What Needs to Happen?
1. Request	What entities need to be extracted from user input?
2. Routing	Which tools or agents are needed for this task?
3. Reasoning	What's the step-by-step plan to accomplish this?
4. Tool Invocation	What specific API calls need to be made?
5. Memory	What information should be stored for future use?
6. Safety	What validations are required before taking action?
7. Response	How should the agent communicate with the user?

Handson-Building a Simple Conversational Agent

innovate achieve lead

Demonstration: Weather Information Agent

A practical example using native OpenAI API in Python

Implementation Stages

1. **Baseline:** LLM without tools - sees limitations
2. **Tool Definition:** Define weather function schema
3. **Tool Selection:** LLM decides when to use tool
4. **Execution:** Call actual weather API
5. **Response Generation:** LLM creates natural answer

Example Interaction

User: "What's the weather like in Mumbai?"

Agent:

1. Extracts location: "Mumbai"
2. Calls weather API
3. Receives: 32°C, Humid, Partly Cloudy
4. Responds: "It's currently 32°C in Mumbai with partly cloudy skies and high humidity."

5.Protocol Landscape for Conversational Agents

As conversational agents become more prevalent, they need **standardized ways to communicate** with tools, data, each other, and UIs.

Why Protocols Matter

Without Standards

- Every vendor has custom API
- Agents can't interoperate
- Vendor lock-in
- Duplicated integration effort

With Standards

- Plug-and-play integrations
- Multi-agent collaboration
- Vendor flexibility
- Ecosystem growth

5. Protocol Landscape for Conversational Agents

innovate achieve lead

MCP (Model Context Protocol) — By Anthropic (2024)

Standardize how LLMs connect to data sources and tools

Agent accessing DBs, files, APIs consistently

A2A Protocol (Agent-to-Agent) — Google / emerging

Standardized inter-agent communication

Multi-agent orchestration (Emerging, 2025)

OpenAI Assistant API — OpenAI

Built-in tools, file access, code interpreter

Rapid agent development with OpenAI infrastructure

Custom REST / GraphQL APIs — Universal

Traditional service integration

Enterprise systems, legacy integrations

LangGraph Protocol — LangChain

Graph-based agent workflows and state management

Complex multi-step reasoning, agent coordination

ANP (Agent Network Protocol) — Emerging

Open standard for peer-to-peer agent discovery and communication across heterogeneous networks

Decentralized agent ecosystems, cross-platform interoperability

Note: The protocol landscape is rapidly evolving. Standards like MCP are emerging, while many production systems still use custom APIs. We'll explore these in detail in Lectures 14-15.

6. Production Concerns for Conversational AI

Building conversational agents that work in development is one thing. Building them to work **reliably at scale** in production is another.

Observability – what to track?

- Track conversation flows
- Monitor tool invocations
- Token usage per conversation
- Response latencies & error traces

Tools: LangSmith, Arize Phoenix, OpenTelemetry

Cost Management – Optimization Strategies

- Prompt caching (50–90% cost reduction)
- Model routing (smaller models when appropriate)
- Token budgets per user/session
- Efficient retrieval (reduce context size)

Example: Customer support ≈ $0.02–0.10 per conversation

Latency Budgets – Target latencies

- Chat responses: < 2 seconds
- Tool execution: < 5 seconds
- Complex reasoning: < 30 seconds
- Set user expectations with progress indicators

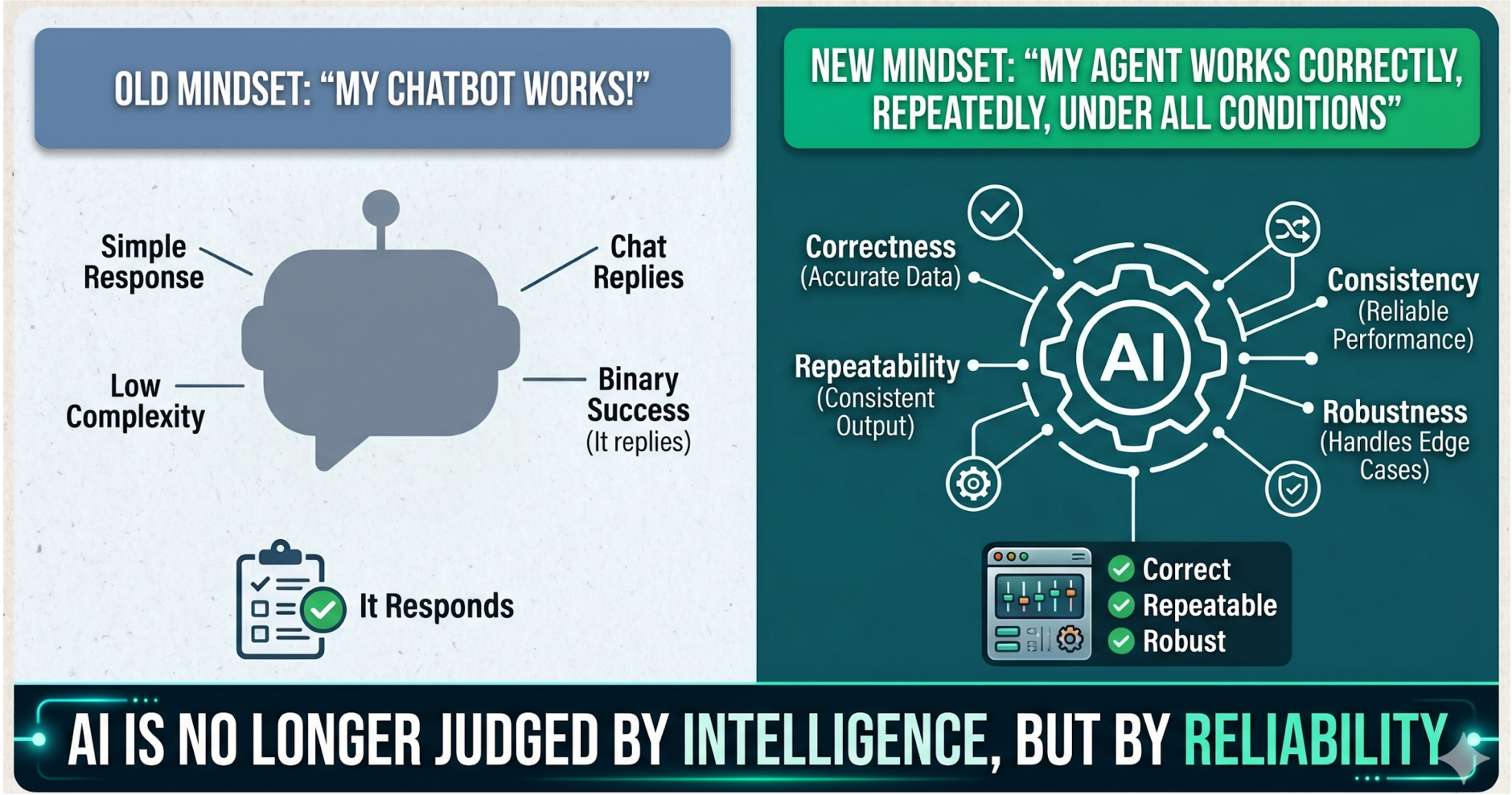
Safety & Security – Defense layers

- Input validation (prompt injection defense)
- PII detection and redaction
- Output filtering (toxicity, hallucinations)
- Human-in-the-loop for critical actions

Principle: Never rely on a single safety layer

6. Production Concerns for Conversational AI

Building conversational agents that work in development is one thing. Building them to work **reliably at scale** in production is another.



Key Takeaways

innovate achieve lead

1 Evolution of Conv. AI

Rule-based (1960s) → ML (2000s) → Deep Learning (2010s) → Transformers (2017) → LLMs (2020) → Agentic AI (2023+)

2 Architecture Shift

Traditional: Intent → Dialogue Manager → Template
Modern: LLM + Tools + Memory + Planning + Safety

3 Components Matter

Modern systems need: NLU, Dialogue Management, Knowledge Access, Action Execution, Response Generation, Memory

4 LLMs Are the Brain

But they need external systems for tools, memory, real-time data, and action execution to be truly useful

5 Agent Lifecycle

Request → Routing → Reasoning → Tool Invocation → Memory → Safety → Response

6 Production Is Different

Observability, cost optimization, latency management, and safety are non-negotiable in real systems

References & Further Reading

Tokenization & LLMs

- Sennrich, R. et al. (2016). Neural Machine Translation of Rare Words with Subword Units. ACL 2016.
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017.
- Brown, T. et al. (2020). Language Models are Few-Shot Learners (GPT-3). NeurIPS 2020.

Agentic AI & Architectures

- Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
- Chase, H. et al. LangChain / LangGraph Documentation. langchain.com (2024).
- Wang, L. et al. (2024). A Survey on Large Language Model based Autonomous Agents. Frontiers of Computer Science.

RAG & Memory

- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Karpukhin, V. et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. EMNLP 2020.

Protocols & Production

- Anthropic. (2024). Model Context Protocol (MCP) Specification. anthropic.com/mcp
- Google. (2025). Agent-to-Agent (A2A) Communication Protocol. ai.google.dev
- Bommasani, R. et al. (2021). On the Opportunities and Risks of Foundation Models. arXiv:2108.07258.

Next Session

innovate achieve lead

Lecture 2: Embeddings, Vector Search & Hybrid Retrieval

Topics

- Deep dive into embeddings and semantic search
- Vector database architecture (HNSW, ANN)
- BM25 + Dense retrieval + RRF (hybrid search)
- Building a practical retrieval system
- Evaluation metrics for retrieval

How to Prepare

- Pre-read: 'Dense Passage Retrieval for Open-Domain QA' (Karpukhin et al., 2020)
- Set up: Python env with OpenAI / Anthropic API keys
- Install: openai, chromadb, rank-bm25
- Review: Today's slides on Canvas

Questions & Discussion

innovate achieve lead

Topics for Discussion:

- Conversational AI use cases in your organization
- Evolution and current state of the field
- Technical challenges you foresee
- Clarifications on any concepts
- Course logistics and expectations

Thank you!

References

BPE : https://huggingface.co/learn/llm-course/en/chapter6/5